Module 4: Human Contribution Beyond AI

Architecture, systems thinking, and domain judgment

Module 4: Human Contribution Beyond AI

Primary sources: Osmani, Chapter 4 selected sections; Thomas & Hunt, Topics 28-36.

Learning Outcomes

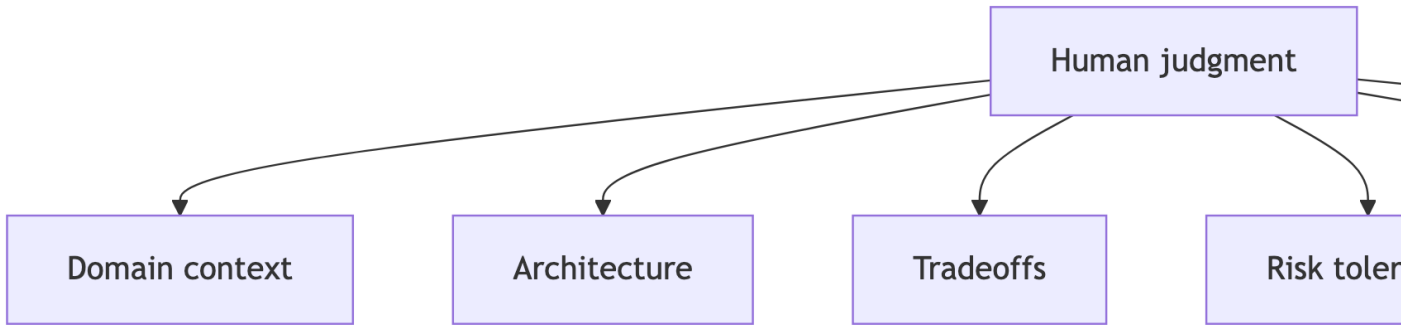
- Identify where human judgment is critical.
- Analyze the role of system design and architecture.
- Explain how domain knowledge impacts output quality.
- Evaluate AI limitations in complex engineering tasks.

The Human 30 Percent

Osmani frames the engineer's durable value as the work AI cannot reliably infer.

- What matters in this domain?
- What tradeoff is acceptable?
- What future change should the design tolerate?
- What risk is not visible in the code?

Where Human Judgment Concentrates



Architect and Editor

AI can draft local code quickly.

Humans still own:

- Boundaries.
- Naming.
- Abstractions.
- Data flow.
- Operational constraints.
- Long-term maintainability.

Systems Thinking

Osmani emphasizes that software changes live inside larger systems.

Humans reason about:

- How one change affects another component.
- Why the current system evolved this way.
- Which constraints come from users, teams, policy, or production.
- Whether an AI shortcut conflicts with long-term system goals.

Code Quality Is Human Oversight

AI-generated code still needs a reviewer who understands the spec and the system.

Look for:

- Edge cases and failure paths.
- Security, privacy, and accessibility implications.
- Inconsistent patterns or naming.
- Tests that prove behavior rather than just coverage.
- Simpler designs that preserve maintainability.

Bend or Break

Chapter 5 argues that code must stay loose enough to change: decouple concepts, handle events deliberately, prefer flexible transformations, avoid inheritance traps, and move changeable details into configuration.

For AI-generated code, watch for:

- Tight coupling.
- Hidden global state.
- Over-specific implementations.
- Missing seams for tests.
- Configuration hardcoded into logic.

The question is not just "does it work?" It is "will this bend when requirements move?"

Context Is a Design Input

AI performs better when given project context, but context is never complete.

Human engineers supply:

- Domain vocabulary.
- Nonfunctional requirements.
- Organizational constraints.
- Legacy system knowledge.
- User consequences.

Concurrency as a Cautionary Case

Concurrent systems reveal AI limits because correctness depends on interactions over time.

Thomas and Hunt distinguish concurrency from parallelism: concurrency is code acting as if parts run at the same time; parallelism is actually running at the same time.

Ask:

- What state is shared?
- What order assumptions exist?
- What happens under failure?
- Can tests expose race conditions?

Architecture Prompt Pattern

Ask AI for alternatives, not the answer.

Useful structure:

- "Generate three designs."
- "List tradeoffs for reliability, testability, and complexity."
- "Identify assumptions."
- "Recommend one for this course scope."
- "Show what would make that recommendation wrong."

Lab 2 Final Connection

Your eval-driven lab should demonstrate human judgment through:

- Measurable criteria.
- Runnable checks.
- Regression comparison.
- Documentation explaining what the tests prove and do not prove.

Practice Activity

Review AI-generated Java code and mark:

- Local correctness issues.
- Design issues.
- Missing tests.
- Hidden assumptions.
- Questions you would ask before merging.

Takeaways

- AI can accelerate implementation but cannot own architecture.
- Human value concentrates in judgment, context, and verification.
- Flexible design protects you from fast wrong turns.